

Carta Intelligence MCP Server

Endpoint & Platform Import Specification — Production Architecture v1.1

Author: Ankur Kumar

Team: Elevation AI

Date: June 15, 2026

Status: Planned

32

MCP Tools

3

MCP Resources

12

DB Tables

11

Canonical
Models

15

Platform Objects

1. Architecture Overview

```
Carta Browser Extraction (Playwright + CDP)
  ↓
Normalization Pipeline (Python)
  ↓
PostgreSQL (canonical multi-tenant tables)
  ↓
FastAPI + MCP Server (port 8082)
  ↓
Platform / LLM Agents / Claude Desktop
```

Key Principle: MCP tools query PostgreSQL only. They never touch raw extraction files directly. All data is normalized, deduplicated, and tenant-isolated before serving.

2. Security & Governance

Layer	Implementation
Authentication	API Keys via <code>X-API-Key</code> header → mapped to <code>tenant_id</code>
Multi-Tenancy	Every query includes <code>AND tenant_id = \$tenant</code> ; tenant derived from API key, never from input
Rate Limiting	100 req/min per IP · 1,000 req/min per tenant · Max 5 concurrent extraction jobs
Soft Delete	Every query includes <code>AND deleted_at IS NULL</code>
Audit Logging	All writes logged: <code>timestamp, tenant_id, api_key_id, tool_name, duration_ms</code>

**Entity
Resolution**

UUID parse → external_id lookup → fuzzy name match (pg_trgm)

3. MCP Resources

URI Pattern	Description
carta://{tenant_id}/entities/funds	Lists all funds available to the tenant
carta://{tenant_id}/entities/companies	Lists all portfolio companies
carta://{tenant_id}/jobs/latest	Latest extraction manifest and success metrics

4. MCP Tools — Complete Specification

4.1 System & Coverage Tools (3)

SYSTEM `get_status`

Health check — verifies MCP server, FastAPI, and PostgreSQL connectivity.

Input: `{}`

Response: `{ "mcp": "running", "api": "reachable", "database": "connected",
"last_extraction": "ISO date" }`

SYSTEM `get_extraction_status`

Returns latest extraction manifest with per-category coverage metrics.

Input: `{}`

Response: `{ "extraction_id", "status", "total_extracted", "total_failed",
"duration_seconds", "coverage": {...} }`

SQL: `SELECT * FROM extraction_jobs WHERE tenant_id = $1 ORDER BY created_at DESC
LIMIT 1`

SYSTEM `get_data_coverage`

Exposes total ingestion and extraction counts across the DB tables to verify ingestion quality and check for gaps.

Input: `{}`

Response: `{ "organizations": 1, "funds": 5, "companies": 104, "investors": 212,
"holdings": 612, "securities": 1240, "valuations": 308, "transactions": 482,
"documents": 3421, "relationships": 1285 }`

SQL: `SELECT (SELECT COUNT(*) FROM organizations WHERE tenant_id = $1) as orgs,
(SELECT COUNT(*) FROM funds WHERE tenant_id = $1) as funds, ...`

4.2 Extraction Tools (2)

POST MCP `download_report`

Downloads/exports a structured data report for a company from Carta. Submits an async task and polls until completion.

Input: `{ "company_name": string (required) }`

Behavior: POST → `/api/download-report` → poll `/api/status/{task_id}` until completed/failed/timeout

POST MCP `download_batch`

Exports Carta data for a list of companies. Submits all tasks in parallel and polls until all complete.

Input: `{ "companies": string[] (required) }`

Response: `{ "total", "success", "failed", "results": [{ "company", "response" }] }`

4.3 Core Entity Tools (9)

GET MCP `get_organization`

Returns the firm/GP organization profile with fund and company counts.

Input: `{}`

Response: `{ "id", "name", "admin_name", "admin_email", "admin_title", "fund_count", "company_count" }`

Errors: `DATA_NOT_LOADED` if no org exists

GET MCP `get_entity`

Generic entity resolver — returns any entity by ID across the entire registry with its general details.

Input: `{ "entity_id": string (required), "entity_type": "Organization|Fund|PortfolioCompany|Investor|Person" (optional) }`

Response: Mapped entity dictionary containing fields, type, metadata, and relations.

GET MCP `list_funds`

Lists all funds, SPVs, GP entities managed by the firm.

Input: `{ "fund_type": "all|Fund|SPV|GPEntity|FundFamily" (default: all), "limit": 50, "offset": 0 }`

Response: `{ "total", "funds": [{ "id", "name", "fund_type", "currency" }] }`

GET MCP `get_fund`

Returns detailed fund information including linked companies.

Input: `{ "fund_id": string (required) }` — accepts UUID or external_fund_id

Errors: `ENTITY_NOT_FOUND`

GET MCP `list_companies`

Lists all portfolio companies with optional holdings summary.

Input: `{ "include_holdings": false, "industry": string, "limit": 50, "offset": 0 }`

SQL: `SELECT c.*, h.cash_cost, h.ownership_pct FROM companies c LEFT JOIN holdings h ...`

GET MCP `get_company`

Returns complete company profile with holdings, latest valuation, and contacts.

Input: `{ "company_id": string (required) }` — UUID, external_company_id, or fuzzy name

Resolution: Fuzzy name match via `pg_trgm` similarity if UUID/external_id not found

Errors: `ENTITY_NOT_FOUND` with `closest_matches` from trigram search

GET MCP `list_investors`

Lists all investors/LPs with total ownership.

Input: `{ "sort_by": "ownership|name", "limit": 50, "offset": 0 }`

GET **MCP** **get_investor**

Returns investor profile with all investments.

Input: { "investor_id": string (required) }

GET **MCP** **search_entities**

Fuzzy search across funds, companies, investors by name using pg_trgm similarity.

Input: { "query": string (required), "entity_type": "all|fund|company|investor",
"limit": 10 }

SQL: UNION ALL across funds, companies, investors with `similarity(name, $1) > 0.3`

4.4 Relationship & Graph Tools (3)

GET MCP `get_relationships`

Exposes raw relationships list from the database, allowing filtering by source, target, or relationship type.

Input: { "source_entity_id": string (optional), "target_entity_id": string (optional), "relationship_type": string (optional), "limit": 100 }

Response: [{ "id", "source_entity_id", "source_entity_type", "target_entity_id", "target_entity_type", "relationship_type", "weight" }]

GET MCP `get_neighbors`

Gets the immediate 1-hop inward and outward neighbors of a specific entity. Essential for navigating Investor → Fund → Company connections.

Input: { "entity_id": string (required), "direction": "in|out|both" (default: both) }

Response: { "inbound": [{ "entity_id", "entity_type", "relationship_type", "name" }], "outbound": [...] }

GET MCP `get_entity_graph`

Exposes the full traversal subgraph starting from a node up to a specified depth.

Input: { "entity_id": string (required), "depth": integer (default: 2, max: 4) }

Response: { "nodes": [{ "id", "name", "type" }], "edges": [{ "source", "target", "type", "weight" }] }

4.5 Financial, Equity & Instruction Tools (9)

GET MCP `list_holdings`

Lists all fund holdings across portfolio companies.

Input: { "fund_id": string (optional), "limit": 50, "offset": 0 }

GET MCP `get_holding`

Detailed holding for a specific company including transactions.

Input: { "company_id": string (required) }

GET MCP `get_cap_table`

Returns cap table for a portfolio company — stakeholders, share classes, ownership percentages.

Input: { "company_id": string (required) }

SQL: SELECT s.*, i.name as owner_name, f.name as fund_name FROM securities s LEFT JOIN investors i ... LEFT JOIN funds f ... WHERE s.company_id = \$1 AND s.tenant_id = \$2

GET MCP `get_securities`

Returns all securities/option plans for a company.

Input: { "company_id": string (required), "security_type": string (optional) }

GET **MCP** **get_valuation**

Returns latest valuation for a company (PostMoney, 409A, or ASC820).

Input: { "company_id": string (required), "valuation_type": "PostMoney|409A|ASC820" (optional) }

GET **MCP** **get_valuation_history**

Returns full valuation history (all rounds) for a company.

Input: { "company_id": string (required) }

GET **MCP** **get_fund_financials**

Returns fund-level financial summary (total invested, value, returns).

Input: { "fund_id": string (required) }

GET **MCP** **get_performance**

Returns IRR, multiple, and transaction history for a company.

Input: { "company_id": string (required) }

Response: { "company_name", "irr_percentage", "multiple", "total_invested", "total_proceeds", "transactions": [...] }

GET **MCP** **get_payment_instructions**

First-class PM tool to retrieve wiring instructions and payment details associated with a fund or GP capital calls.

Input: { "fund_id": string (required) }

Response: { "bank_name", "account_number", "routing_number", "swift_code", "beneficiary_name", "reference_notes" }

4.6 Supporting, Document & Communication Tools (5)

GET MCP **list_people**

Lists contacts and team members.

Input: { "company_id": string (optional), "role": string, "limit": 50 }

GET MCP **get_person**

Returns person profile with associated entities.

Input: { "person_id": string (required) }

GET MCP **list_documents**

Lists documents with type and entity filters.

Input: { "entity_id": string, "document_type": string, "limit": 50 }

GET MCP **get_document**

Retrieves metadata for a specific document including document ID, title, type, and associated entity ID.

Input: { "document_id": string (required) }

Response: { "document_id", "title", "document_type", "entity_id", "file_type", "document_date", "created_at" }

GET MCP **download_document**

Returns download URL for a document file.

Input: { "document_id": string (required) }

Response: { "document_id", "download_url", "file_type": "PDF" }

GET MCP **list_activities**

First-class PM tool that lists user log activities, notes, and firm-level communications.

Input: { "entity_id": string (optional), "limit": 50 }

Response: { "activities": [{ "activity_id", "entity_id", "entity_type", "title", "note_body", "created_at" }] }

5. Platform Import Contract & Data Mapping

Mapping from Product Management (PM) Category to Database, MCP Tools, and Ingestion Platform Objects

PM Business Category	DB Table	MCP Tool(s)	Platform Object Name
Firm / GP	organizations	get_organization	OrganizationNode
Fund / Vehicle	funds (type=Fund)	list_funds , get_fund	FundNode
SPV	funds (type=SPV)	list_funds (type=SPV)	SPVNode
Fund Family	funds (type=FundFamily)	list_funds (type=FundFamily)	FundFamilyNode
Portfolio Company	companies	list_companies , get_company	CompanyNode
Investment / Holding	holdings	list_holdings , get_holding	HoldingNode
Investor / LP	investors	list_investors , get_investor	InvestorNode
Cap Table	securities	get_cap_table	CapTableSubgraph
Securities	securities	get_securities	SecurityNode
Valuations / 409A	valuations	get_valuation , get_valuation_history	ValuationNode
Performance Metrics	performance_metrics	get_performance	PerformanceNode
People / Users	people	list_people , get_person	ContactNode
Documents	documents	list_documents , get_document , download_document	DocumentNode
Communications	documents (type=Note)	list_activities	CommunicationNode
Wire Instructions	raw_metadata / custom	get_payment_instructions	WireInstructionNode
Entity Registry	organizations	get_entity	RegistryNode

6. Error Code Catalog

Code	HTTP	Trigger
ENTITY_NOT_FOUND	404	Entity ID or fuzzy name match fails
INVALID_PARAMETERS	400	Missing required field or invalid type
DATA_NOT_LOADED	503	PostgreSQL empty — normalization not run
EXTRACTION_REQUIRED	503	No extraction data exists for entity
PARTIAL_DATA	206	Some fields missing due to extraction failures
DB_ERROR	500	PostgreSQL connection or query failure
TENANT_UNAUTHORIZED	403	API key not mapped to requested tenant
RATE_LIMITED	429	Exceeded request quota

7. Validation Rules (All Tools)

Rule	Implementation
Entity ID resolution	UUID parse → external_id lookup → fuzzy name match (pg_trgm)
Tenant isolation	Every query: <code>AND tenant_id = \$tenant</code>
Soft delete filter	Every query: <code>AND deleted_at IS NULL</code>
Pagination	Default limit=50, max limit=200
Date format	ISO 8601 in responses; accept MM/DD/YYYY in input
Currency	Always include <code>currency</code> field with ISO 4217 code

8. Database Schema Summary

Table	Unique Key	Primary Indexes
organizations	(tenant_id, external_org_pk)	tenant_id
funds	(tenant_id, external_fund_id)	tenant_id, name_trgm
companies	(tenant_id, external_company_id)	tenant_id, name_trgm
investors	(tenant_id, external_investor_id)	tenant_id, name_trgm

holdings	(tenant_id, fund_id, company_id)	tenant_id, company_id
securities	(tenant_id, external_security_id)	tenant_id, company_id
valuations	—	tenant_id, (company_id, valuation_date DESC)
transactions	—	tenant_id, (company_id, transaction_date DESC)
documents	—	tenant_id, (entity_id, entity_type)
people	(tenant_id, email)	tenant_id, company_id
relationships	(tenant_id, source, target, type)	source_entity_id, target_entity_id
performance_metrics	—	tenant_id, company_id